Does Steepest Selection Help?

An Empirical Study of Saddle-Point-Escaping Optimizers on Non-Convex Machine-Learning Loss Landscapes

Shyam Patadia

https://github.com/shyampatadia/spgd_research

Worcester Polytechnic Institute
MA 551 Computational Statistics

May 4, 2026

Roadmap

1. The optimisation problem at the heart of ML, and why saddles matter.
2. SPGD — what it is, the algorithm, what's novel.
3. The gap we close: what the original authors did *not* test.
4. Our research design and the missing control (RPGD).
5. Four discriminating experiments. What each tells us.
6. Where the algorithm helps, and where it doesn't.
7. How our work compares to the original SPGD paper.
8. Conclusion + take-aways.

Training a model is an optimisation problem

The empirical risk

$$\min_{\theta \in \mathbb{R}^d} f(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(\theta; x_i, y_i)$$

- For modern architectures, d is in the millions and f is non-convex.
- First-order methods (SGD, Adam) use only ∇f.
- In high dimensions the critical-point structure is dominated by saddle points, not local minima dauphin2014saddle, choromanska2015loss.
- Near a saddle, $\|\nabla f\| \to 0$ but the iterate is not a minimiser $\Rightarrow$ the optimiser stalls.

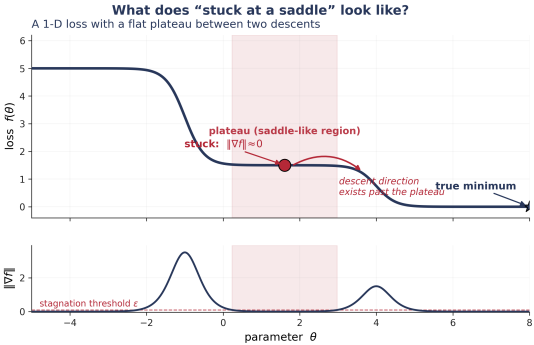
What does “stuck at a saddle” actually look like?

Picture a literal horse’s saddle with a ball placed on it.

Roll the ball *front-to-back* — along the dip where the rider would sit — and it swings back and forth, eventually settling in the middle. That middle point is the saddle point: the loss looks minimised in this direction.

But the ball is in *unstable equilibrium.* Nudge it *perpendicular* to that dip — toward either flank of the saddle — and it falls off drastically. The surface curves sharply *down* in that direction.

So at a saddle point: gradient is zero (looks like “done”), but a real descent direction exists — and a first-order method can’t see it. The bottom panel below shows the consequence in time: $\|\nabla f\|$ drops below the stagnation threshold ε and stays there.



1-D cross-section through the saddle’s stable direction (top); resulting gradient-norm trace (bottom).

Two ways to get unstuck — intuitively

Imagine a recommender system that's stuck

Spotify keeps suggesting the same kind of song to a user who keeps skipping. The system's best guess is no longer improving — it's *stuck*. Two strategies to break out:

Adaptive: tune what you already know

Look at history feature-by-feature: the user always finishes acoustic songs (small, careful steps in that direction), always skips electronic ones (big steps to avoid that region). Scale each direction by how confident you are.

Adam, RMSProp, AdaGrad do exactly this for gradient descent: rescale ∇f per-coordinate by past gradient magnitudes.

Still only uses information you've already collected.

Perturbation: play a wildcard

Just *recommend a random song* from the whole catalogue — even one that doesn't fit the user's pattern. Most get skipped. But occasionally, by accident, one is a banger they never would have found from their history alone.

PGD jin2017escape injects Gaussian noise when the gradient gets small, then continues with gradient descent.

Commits to the random pick regardless of where it lands.

Why does "take a leap" actually work?

The geometry behind it

A strict saddle has a "tail" — directions of *negative* curvature where the loss decreases. The gradient is zero *exactly* at the saddle, so first-order methods see nothing.

But: a random direction will, with positive probability, have a component along that tail. Once you've nudged onto the tail, the gradient is again informative and you slide downhill.

- In 1-D: a flat plateau followed by a slope. Random jump left or right $\Rightarrow$ probability $\frac{1}{2}$ you land on the slope.
- In high d: tail directions occupy a measure-positive fraction of the unit sphere; random isotropic noise hits one with overwhelming probability.

Why does "take a leap" actually work?

The geometry behind it

A strict saddle has a "tail" — directions of *negative* curvature where the loss decreases. The gradient is zero *exactly* at the saddle, so first-order methods see nothing.

But: a random direction will, with positive probability, have a component along that tail. Once you've nudged onto the tail, the gradient is again informative and you slide downhill.

- In 1-D: a flat plateau followed by a slope. Random jump left or right $\Rightarrow$ probability $\frac{1}{2}$ you land on the slope.
- In high d: tail directions occupy a measure-positive fraction of the unit sphere; random isotropic noise hits one with overwhelming probability.

The remaining question

Among the random jumps you could take, which one should you commit to?

This is precisely the gap SPGD fills.

SPGD, intuitively — "look before you leap"

The one-line idea

PGD takes *one* random jump and hopes for the best.

SPGD takes N_P random jumps, and commits to the one that actually lowers the loss the most.

Why this should help

A single random jump might land you back on the plateau, or even *up* the slope of the saddle's tail.

Out of N_P candidates, you're far more likely to find one that points to a genuinely lower loss — and you only commit if that minimum candidate actually beats your current position.

Where SPGD sits between two extremes

Plain GD	⟵ SPGD ⟶	Random search
directed		exploratory
(no exploration)		(no direction)

Directed efficiency of GD with the exploratory benefits of stochastic search.

SPGD, intuitively — "look before you leap"

The one-line idea

PGD takes *one* random jump and hopes for the best.

SPGD takes N_P random jumps, and commits to the one that actually lowers the loss the most.

Why this should help

A single random jump might land you back on the plateau, or even *up* the slope of the saddle's tail.

Out of N_P candidates, you're far more likely to find one that points to a genuinely lower loss — and you only commit if that minimum candidate actually beats your current position.

Where SPGD sits between two extremes

Plain GD	⟵ SPGD ⟶	Random search
directed		exploratory
(no exploration)		(no direction)

Directed efficiency of GD with the exploratory benefits of stochastic search.

Now the technical algorithm just makes this idea precise.

Now the precise version — three differences from PGD

vahedi2024spgd

Three differences from PGD:

1. When? Perturbation fires *unconditionally* every *IterP* steps, not gated on stagnation detection.
   (PGD waits until $\|\nabla f\|$ is small; SPGD perturbs proactively.)
2. How many? N_P candidates per phase, not one.
   (PGD: single Gaussian sample. SPGD: N_P uniform samples.)
3. Which one? Commit the candidate with the steepest loss decrease (paper's $\leq$ rule); else take the gradient step.
   (PGD has no selection step at all.)

The SPGD algorithm

Require: initial θ_0, learning rate η, amplitude *Amp*, candidates N_P, interval *IterP*, total steps T

for $t = 0, \ldots, T-1$ **do**
  if $t \bmod \mathit{IterP} = 0$ **and** $t > 0$ **then**
    Sample N_P candidates $\delta^{(k)} \sim \mathrm{Uniform}(-\mathit{Amp}, \mathit{Amp})^d$
    Evaluate $f(\theta_t + \delta^{(k)})$ for $k = 1, \ldots, N_P$
    $k^\star \leftarrow \arg\min_k f(\theta_t + \delta^{(k)})$
    if $f(\theta_t + \delta^{(k^\star)}) \le f(\theta_t)$ **then**
      $\theta_t \leftarrow \theta_t + \delta^{(k^\star)}$ ▷ *accept candidate (paper's $\le$ rule)*
    end if
  end if
  $\theta_{t+1} \leftarrow \theta_t - \eta \nabla f(\theta_t)$ ▷ *standard gradient step*
end for

- Cost per perturbation phase: N_P extra forward passes (no extra gradients).
- Hyperparameters that matter: N_P (exploration breadth) and *IterP* (exploration frequency).

What did the original SPGD paper test?

Tested on

- 4 smooth 2-D benchmark functions (Peaks, Ackley, Easom, Levy 13)
- A 3-D component packing problem (NP-hard, mechanical engineering)

Compared against

- Plain Gradient Descent (GD)
- PGD
- Simulated Annealing
- MATLAB `fmincon`

What the paper does not include:

- Any neural-network experiment.
- Any same-compute random-selection control (so we cannot tell whether the *selection rule* is what matters).

What did the original SPGD paper test?

Tested on

- 4 smooth 2-D benchmark functions (Peaks, Ackley, Easom, Levy 13)
- A 3-D component packing problem (NP-hard, mechanical engineering)

Compared against

- Plain Gradient Descent (GD)
- PGD
- Simulated Annealing
- MATLAB `fmincon`

What the paper does not include:

- Any neural-network experiment.
- Any same-compute random-selection control (so we cannot tell whether the *selection rule* is what matters).

The paper's own future-work claim

"*Potential to significantly enhance neural network training*" [Conclusion]vahedi2024spgd

This claim has not been tested. That is the gap we close.

Two questions only experiments can answer

Q1 — Does the paper's NN-training claim hold?

The authors say SPGD "has potential" on neural networks. Does it? Under what conditions, by how much?

Q2 — Is it the selection rule or just the perturbation?

SPGD's mechanism has two parts: multi-candidate perturbation and steepest selection. The original paper conflates them. We need a control that perturbs the same way but *selects randomly*.

Two questions only experiments can answer

Q1 — Does the paper's NN-training claim hold?

The authors say SPGD "has potential" on neural networks. Does it? Under what conditions, by how much?

Q2 — Is it the selection rule or just the perturbation?

SPGD's mechanism has two parts: multi-candidate perturbation and steepest selection. The original paper conflates them. We need a control that perturbs the same way but *selects randomly*.

Our central methodological contribution is to introduce that control.

The five optimisers we compare

SGD	Plain gradient descent. The first-order baseline.
Adam	Per-parameter adaptive scaling. The modern ML default.
PGD	Gradient descent with one Gaussian perturbation when $\|\nabla f\|$ is small.
RPGD	Random control: same multi-candidate perturbation as SPGD, but commits a *random* candidate (not the steepest).
SPGD	Steepest of N_P uniform candidates committed; paper's $\leq$ rule.

What each contrast isolates

- SPGD vs. RPGD: is the *selection rule* doing useful work?
- SPGD vs. PGD: does multi-candidate perturbation help at all?
- SPGD vs. SGD: does perturbation+selection beat no perturbation?

Paired-seed protocol + direct saddle diagnostics

Paired-seed protocol

For each (experiment, seed):

- data split, init,
- all per-seed randomness

are fixed *before* the optimiser is selected.

All five optimisers run on *identical* starting state.

Differences are paired, not unpaired.

Direct saddle diagnostics

- *Stagnation episode*: max consecutive run with $\|\nabla f\|_2 < \varepsilon$.
- $\varepsilon = 10^{-4}$ on smooth benchmarks; 10^{-3} on noisy mini-batches.
- *Escape-time*: first step at which loss drops 5% below initial. Reported where saddle plateaus exist.

We measure saddle behaviour directly — not infer it from loss curves.

Six experiments (four discriminating, two diagnostic)

Exp	What it tests	Role
1	Synthetic Rastrigin / Ackley / Rosenbrock at $d \in \{2, 10, 50\}$	main
3	OpenML-CC18 tabular: 5 datasets, 3-layer MLP	main
4	CIFAR-10 / ResNet-18 anchor (paired SPGD vs. RPGD)	main
6	MovieLens-100K matrix completion (Burer–Monteiro)	main
2	Two Moons — visualisation sanity check	appendix
5	$N_P \times IterP$ ablation	appendix

Increasing scale: $d = 2 \;\rightarrow\; 1.3{\times}10^4 \;\rightarrow\; 1.1{\times}10^7$ *parameters.*

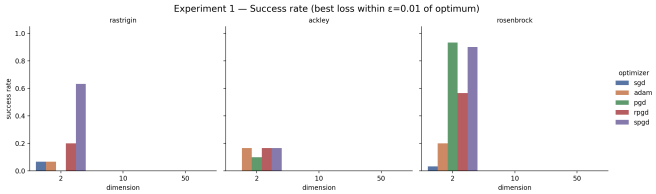
Experiment 1 — Synthetic benchmarks (setup)

Three benchmark functions.

- **Rastrigin:** egg-carton — hundreds of local minima around one global. Pure *escape* problem.
- **Ackley:** funnel-shaped global basin + many shallow pits. Mixed escape/speed.
- **Rosenbrock:** curved banana valley, *no* local minima. Pure *speed* test.

Protocol. $d \in \{2, 10, 50\}$; 30 seeds per cell; success $\equiv f(\theta) < 10^{-2}$ from global optimum.

Why d=10 and d=50 bars are empty: the 10^{-2}-tight target ball is simply unreachable in budget — not failure, just a binary metric saturating. *Loss-value* differences at higher d *do* discriminate (next slide).



Success rate per (function, dimension, optimiser). d=10 and d=50 are zero across the board — see caption-text on left.

Experiment 1 — Per-benchmark interpretation

- Rastrigin (d=2, escape problem): SPGD **63%** vs RPGD **20%** — same compute, same mechanism, only the selection rule differs. Cleanest evidence in this experiment that the steepest-selection rule does real work. SGD/Adam stuck at 7%; PGD at 0% (single Gaussian sample isn't enough to find an escape direction).
- Ackley (d=10, mixed): success rate is uninformative at d=2 (everyone ∼17%). At d=10, **SPGD's mean final loss is** -2.57 **below RPGD's** on a paired comparison — the binary success criterion doesn't fire here, but the loss-value gap is the headline number from this experiment.
- Rosenbrock (d=2, pure speed): no local minima — so this is purely a question of who descends a curved valley faster. PGD **93%** and SPGD **90%** both dominate; RPGD partial at 57%; Adam 20%; SGD 3% (gradient poorly aligned with the curved valley axis).

Caveat carried into all later experiments

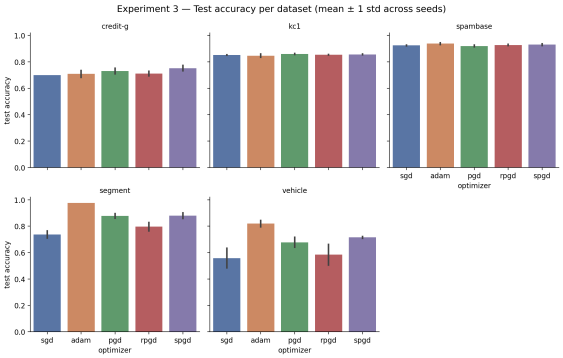
Constant per-coordinate *Amp* becomes destructive at d=50: total displacement scales as $\sqrt{d}$, so an amplitude tuned at d=2 is $\sqrt{50/2} \approx 5\times$ too large at d=50. Per-parameter amplitude scaling ($Amp_p \propto \mathrm{std}(\theta_p^{(0)})$) is used in all subsequent experiments.

Experiment 3 — OpenML-CC18 tabular

Setup. 5 datasets (`credit-g`, `kc1`, `spambase`, `segment`, `vehicle`), 3-layer MLP, 3 seeds each.

Findings.

- SPGD's mean test accuracy above RPGD's on a majority of (dataset, seed) pairs.
- Adam dominates absolute accuracy on all 5 datasets — tabular landscapes are well-conditioned.
- Stagnation rare; only `credit-g` and `vehicle` produced any non-zero counts.



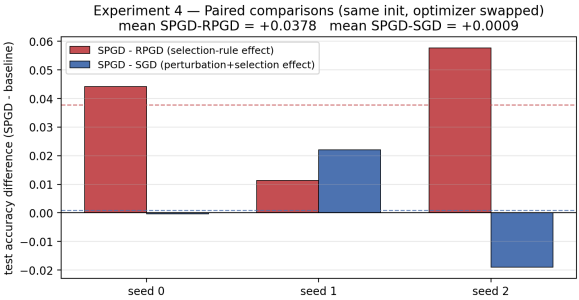
Test accuracy per dataset (mean $\pm$ 1 std, 3 seeds).

Experiment 4 — CIFAR-10 / ResNet-18 (anchor result)

Setup. ResNet-18 (11M params), BN disabled in `layer1`/`layer2` to retain saddle structure; 50 epochs, 3 seeds; $Amp{=}10^{-3}$, $N_P{=}5$, $IterP{=}200$. Run on the WPI Turing cluster (A30).

Final test accuracies (3 seeds).

Adam	0.9015 ± 0.014
SPGD	0.8277 ± 0.028
SGD	0.8268 ± 0.019
PGD	0.8091 ± 0.014
RPGD	0.7899 ± 0.046



Per-seed paired deltas SPGD − RPGD and SPGD − SGD.

Headline

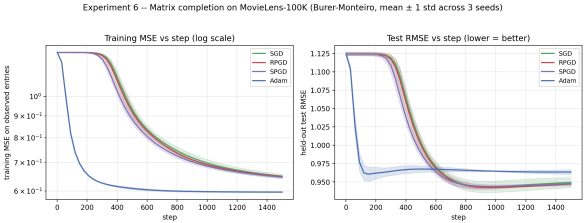
SPGD beats RPGD on all three seeds: paired mean +3.78 pp test accuracy (+4.42, +1.14, +5.77). Cleanest evidence in the study that the *selection rule* matters.

Experiment 6 — MovieLens matrix completion (saddle escape)

Why this benchmark. Burer–Monteiro factorisation of a 943×1682 rating matrix at rank 5, init $\mathcal{N}(0, 0.01^2)$. Documented saddle near origin that gradient descent escapes only slowly ge2016matrix, sun2018geometric.

The two key numbers.

- Acceptance rate: SPGD fires at $30 \pm 2\%$; RPGD at $9 \pm 1\%$ $(3\times)$.
  The selection rule is empirically active.
- Escape-time (first 5% loss drop):
  SPGD−SGD -50 steps (3/3 seeds)
  SPGD−RPGD -40 steps (3/3 seeds)



Train MSE (left) and test RMSE (right). SPGD leads SGD through the saddle, lead consumed by convergence.

The split result

Mechanism transfers (SPGD fires as designed in $d \approx 13{,}000$), but the optimisation lead does not translate into a held-out RMSE gap (paired $\Delta = -0.002$, sign mixed).
Mechanism-positive, generalisation-neutral.

Cross-experiment summary

Setting	SPGD vs. RPGD	Stagnation?	Effect size	Read
Synthetic (d=10, Ackley)	SPGD ↓	yes	paired −2.57 loss	rule helps
OpenML tabular	SPGD slightly ahead	rare	∼1 pp acc	rule helps
CIFAR-10 / ResNet-18	SPGD > RPGD on 3/3	none	+3.78 pp	rule helps strongly
MovieLens MC	training: SPGD ↓	yes (saddle)	escape −50 steps	rule fires

Headline result, one line

Across the four discriminating experiments, the steepest-selection rule provides a measurable advantage over same-mechanism random perturbation.

Cross-experiment summary

Setting	SPGD vs. RPGD	Stagnation?	Effect size	Read
Synthetic (d=10, Ackley)	SPGD ↓	yes	paired −2.57 loss	rule helps
OpenML tabular	SPGD slightly ahead	rare	∼1 pp acc	rule helps
CIFAR-10 / ResNet-18	SPGD > RPGD on 3/3	none	+3.78 pp	rule helps strongly
MovieLens MC	training: SPGD ↓	yes (saddle)	escape −50 steps	rule fires

Headline result, one line

Across the four discriminating experiments, the steepest-selection rule provides a measurable advantage over same-mechanism random perturbation.

The advantage is mechanism-positive on every setting,
and generalisation-positive on CIFAR-10 specifically.

Where the algorithm doesn't help

1. The saddle-escape framing is over-stated for modern ML

Zero stagnation episodes ($\|\nabla f\| < 10^{-3}$ for ≥ 5 steps) on *all 15 CIFAR-10 runs*, despite our deliberately disabling BN in the early layers to preserve saddle structure. SPGD's CIFAR-10 advantage therefore cannot be attributed to plateau escape per se.

2. Generalisation gain on matrix completion is null

SPGD's training-time lead on MovieLens does not survive to held-out RMSE (paired $\Delta = -0.002$, sign mixed across seeds).

3. Adam dominates absolute accuracy throughout

Per-parameter adaptive scaling implicitly conditions the loss landscape; none of our perturbation methods rescale the gradient. The right comparison is therefore SPGD vs. same-compute random control, not SPGD vs. Adam.

What did we add over the original SPGD paper?

	Original (Vahedi & Ilies, 2024)	This work
Domain	2-D test functions, 3-D engineering packing	Neural networks & matrix completion
Compared with Random control?	GD, PGD, SA, `fmincon` absent	SGD, Adam, PGD, RPGD, SPGD introduced (RPGD)
Saddle diagnostic	inferred from loss curves	stagnation episodes + escape-time
Paired protocol	no	yes (across all experiments)
Compute scale	seconds, 2-D	up to ResNet-18 / CIFAR-10

Bottom line

The original paper proposes the algorithm; we provide the first controlled, paired, ML-scale test of its central claim — and the only same-compute random-selection ablation in the literature.

Three things this study contributes

1. The first ML-scale test of SPGD.
   The original paper names neural-network training as an in-scope use case (*"unconstrained problems such as 2-D test function benchmarks and neural network training..."*) but reports no NN experiment. Our four discriminating experiments span synthetic, tabular, vision, and matrix completion at up to 1.1×10^7 parameters.
2. An RPGD control that isolates the selection rule.
   Without a same-compute random-selection baseline, "SPGD beats baselines" is confounded with "perturbation alone helps." On CIFAR-10/ResNet-18 the SPGD-vs-RPGD paired delta is $+3.78$ pp on every seed; that gap could not have been measured by the original protocol.
3. Bounded scope claims, supported by direct diagnostics.
   Stagnation episodes and escape-time give us measurements, not assertions: the algorithm helps where the selection rule fires (CIFAR-10, d=10 Ackley), and is neutral where it doesn't transfer to held-out loss (MovieLens RMSE).

What we conclude

Main empirical claim

On modern ML loss landscapes, SPGD's value is best framed as a biased candidate-selection rule, not a saddle-escape mechanism.

Methodological recommendation

Future perturbation-based optimisers should be benchmarked against a same-compute random-selection control, a comparison missing from the existing literature. On our deepest setting it is the selection *rule*, not perturbation alone, that does the useful work.

One-line take-away

It's not the noise. It's the choice.

Limitations (briefly)

- Three seeds at the CIFAR-10 scale — mitigated by paired protocol but unpaired stds are wide.
- Stagnation threshold ε is fixed; a relative threshold might surface plateaus our absolute one misses.
- No momentum / lr-schedule on SPGD — adaptivity held out to isolate the selection rule.
- Constant amplitude *Amp*; per-parameter scaling $Amp_p \propto \mathrm{std}(\theta_p^{(0)})$ would likely improve d=50 and ResNet-18.
- Flat-minima / sharpness measurement scoped out (zero-stagnation made it peripheral here).

Thank you.

Questions?

Repository → `https://github.com/shyampatadia/spgd_research`

Code, configs, and figures → `src/spgd_study/`, `experiments/`, `figures/`.

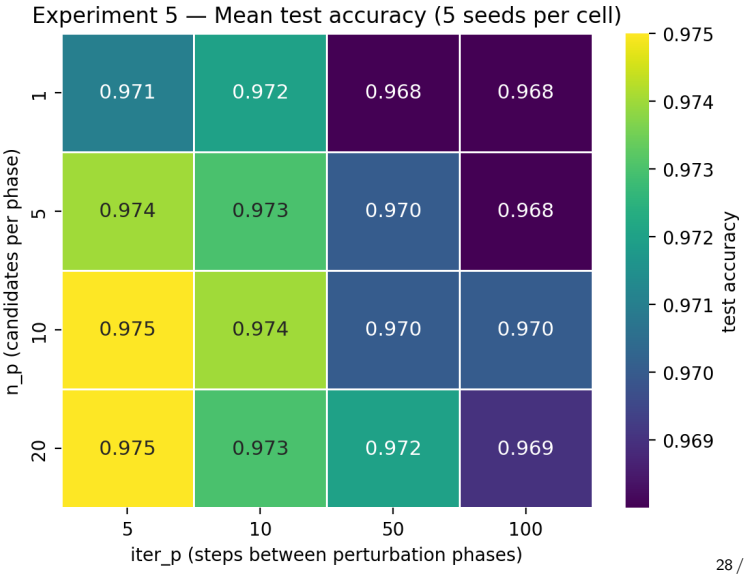
Numerical artefacts → `results/exp6_escape_time_paired.json` (escape-time deltas).

Backup — Why $\varepsilon = 10^{-3}$ on noisy gradients?

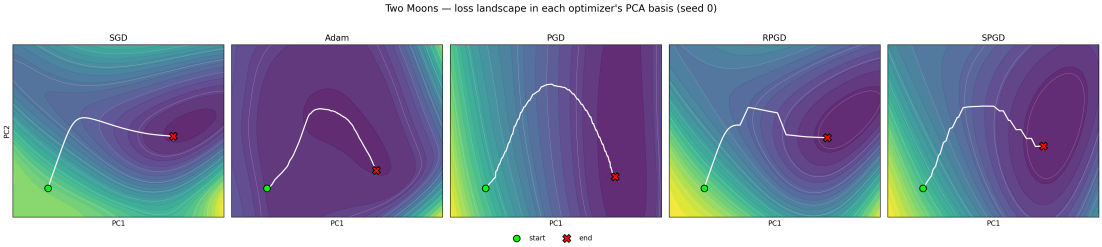
- On smooth full-batch losses (Exp 1, Two Moons), gradient norms decay smoothly to $O(10^{-4})$ near minima; we use the proposal's $\varepsilon = 10^{-4}$.
- On mini-batched CIFAR-10/OpenML, the gradient-norm time series is dominated by mini-batch noise — typical norms are $O(10^{-2})$. A 10^{-4} threshold would never fire.
- We pick 10^{-3} as the consistent "well below typical mini-batch noise floor" threshold.
- The *qualitative* CIFAR-10 result (zero stagnation) is threshold-dependent; we flag this as a limitation.

Backup — $N_P \times$ IterP ablation (appendix)

- Grid: $N_P \in \{1, 5, 10, 20\}$, $IterP \in \{5, 10, 50, 100\}$.
- 80 runs (5 seeds$\times$16 cells), Two Moons, $\sim$2 min total.
- Compute scales $\propto N_P/IterP$, range $\sim 400\times$.
- Accuracy heatmap mostly flat: SPGD is robust to hyper choice.
- Loss heatmap: smaller $IterP$ reaches lower loss without accuracy gain $\Rightarrow$ broader basins, not deeper ones.



Backup — Two Moons trajectories (appendix)



Shared PCA basis: PC1, PC2 mean the same in every panel; θ_0 at the origin. Adam moves substantially along PC1 (adaptive directions); SGD/PGD/RPGD/SPGD all move predominantly along PC2.

Diagnostic of geometry, not a benchmark of accuracy.

Backup — Bibliography (selected)

A. M. Vahedi, H. T. Ilies. "SPGD: Steepest Perturbed Gradient Descent Optimization," *arXiv:2411.04946*, 2024.

C. Jin, R. Ge, P. Netrapalli, S. M. Kakade, M. I. Jordan. "How to Escape Saddle Points Efficiently," *ICML*, 2017.

R. Ge, F. Huang, C. Jin, Y. Yuan. "Escaping from saddle points," *COLT*, 2015.

Y. Dauphin et al. "Identifying and attacking the saddle point problem in high-dimensional non-convex optimization," *NeurIPS*, 2014.

R. Ge, J. D. Lee, T. Ma. "Matrix completion has no spurious local minimum," *NeurIPS*, 2016.

J. Sun et al. "A geometric analysis of phase retrieval and related problems." 2018.

K. He, X. Zhang, S. Ren, J. Sun. "Deep residual learning for image recognition," *CVPR*, 2016.

D. Kingma, J. Ba. "Adam: A method for stochastic optimization," *ICLR*, 2015.